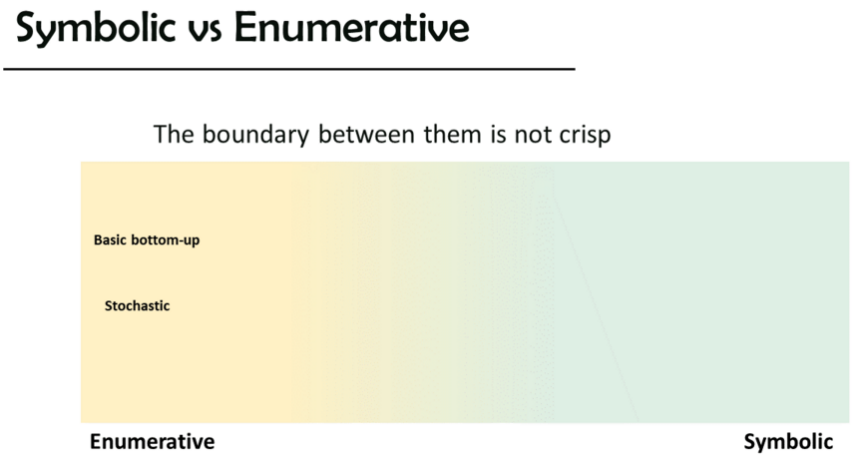
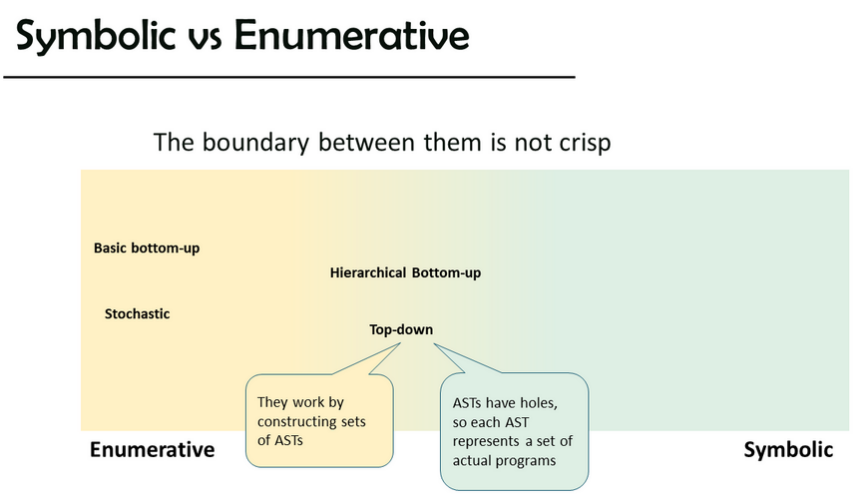
Lecture 6

Version Space Algebras from SMARTedit to FlashFill.

Up to this point, we have been exploring a series of enumerative search techniques; what made these techniques enumerative was that we were explicitly constructing ASTs one-by-one in the process of exploring the space.

For the rest of this unit, we will be switching gears to a different class of search techniques based on *symbolic* representations of program spaces. The key idea behind these techniques is that instead of enumerating ASTs one-by-one we have some data-structures (or symbols) that concisely represent entire sets of programs. By manipulating these symbols, we can efficiently eliminate large sets of possible programs.

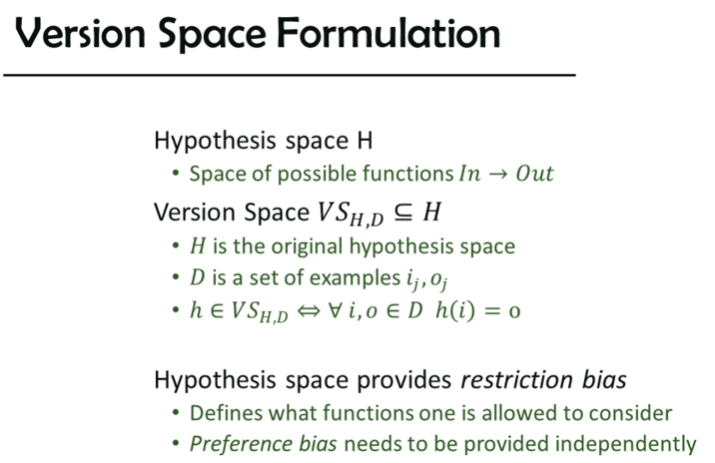




It is worth emphasizing that the distinction between enumerative and symbolic search techniques is not as crisp as it may appear at first sight. In fact, while we have been focusing so far on enumerative techniques, the reader may have noted that some of the techniques we have covered contain already some symbolic aspects. The basic bottom-up search and the stochastic search strategies were very clearly enumerative. Both techniques construct and evaluate complete ASTs one at a time before accepting or rejecting them. However, the Hierarchical bottom-up techniques and the top-down search do have some symbolic aspects to them. In the hierarchical search we described in Lecture 3, we were performing enumeration over a language of queries with holes, where each query with a hole actually represented an entire space of queries. Similarly, in the top-down search, each query with an un-expanded non-terminal actually represented the whole set of possible expansions.

In this lecture, we will be covering a class of techniques that also sit close to the boundary, but which rely heavily on the ability to compactly represent very large sets of programs and to manipulate those representations in order to get a concrete program.

6.1 Version Spaces



We start the lecture with a discussion of the Version Space approach to inductive synthesis as described by Lau, Wolfman, Domingos, Weld on their paper on programming by demonstration [1]. In this paper, the space of programs is called a *Hypothesis Space* (H), and given a dataset of inputs and outputs $D = \{(in_i, out_i)\}$, a *Version Space* $VS_{H,D}$ is the subset of H corresponding to programs that satisfy the examples in the given dataset.

A *Version Space Algebra* (VSA) defines a set of operations that allow us to manipulate and compose version spaces. What really characterizes the VSA-based approaches is the use of compact symbolic representations for the version spaces, which allow us to manipulate them efficiently.

6.2 Lattice-based version spaces

One important class of representations are Lattice-based representations. At a high-level, a lattice is a partially ordered set where we can define a unique least upper bound (lub) and a greatest lower bound (glb) for any pair of elements x and y, where a least upper bound is the smallest element that is greater than or equal to x and y and the greatest lower bound is the largest element that is less than or equal to x and y. For example, the integers form a lattice, where $lub(x, y) = max(x, y)$ and $glb(x, y) = min(x, y)$. Another example of a lattice is the set of all predicates over some set of variables $p(x)$. We can define a partial order over this set by saying that $p \leq q$ iff $p \Rightarrow q$. Then, given two predicates p and q, we can see that $lub(p, q) = p \vee q$ and $glb(p, q) = p \wedge q$.

Now, one common way of defining a range of integers is by its endpoints, so we can use the notation $[a, b]$ to represent the set of integers $\{x | a \leq x \leq b\}$,

assuming that $a \leq b$. It turns out the same idea can apply to other lattices; given two elements $p, q \in H$ belonging to some lattice H, if $p \leq q$, then we can define the set $[p, q] = \{x \in H | p \leq x \text{ and } x \leq q\}$. In the version space literature, it is said that a version space is *Boundary Set Representable* if the hypothesis space is a lattice and the version space can be represented as a range in terms of its two endpoints $[S, G]$. The letter S is used to refer to the most specific hypothesis, and the letter G to the most general. This idea of using endpoints in a lattice to represent a set of concepts goes back to Mitchell's seminal paper on version spaces back in 1982 [2].

Example: An example from Lau et al. is a class of functions `FindSuffix(T)`. This class of functions is parameterized by a string T, and is used in the text editing domain to move the cursor right before the next occurrence of the string T.

Now, consider the following piece of text from Churchill's famous speech.

> *We shall go on to the end. We ↓1shall fight in France, we ↓2shall fight on the seas and oceans, we ↓3shall fight with growing confidence and growing strength in the air.*

Suppose we move the cursor from the position marked as 1, right before the word shall, to position 2. The movement can be implemented as an application of `FindSuffix(T)` for many different strings `T`, including `T="s"`, `T="shall"`, `T="shall fight on the seas and oceans"` and more.

The important thing to note is that we can define a lattice over the strings T based on the following operations.

$$\begin{array}{ll} T_1 \leq T_2 & \text{iff } T_1 \text{ prefix } T_2 \\ glb(T_1, T_2) = & \text{longest common prefix of } T_1 \text{ and } T_2 \\ lub(T_1, T_2) = & \text{shortest string that has } T_1 \text{ and } T_2 \text{ as prefix} \end{array}$$

The reader may notice that the structure above is not quite a lattice, because the lub operation is not well defined for all inputs. For example, lub("hello","world") is not well defined because there is no string that has both words as a prefix. A common trick to address this problem for lattices is to define a special value $\top$ (pronounced "top") that is defined to be greater than all other values. In this case, $\top$ would be a special string with the property that $\forall T.T$ prefix $\top$. Then it is clear that lub("hello","world") = $\top$.

With this lattice, we can see that the space of functions `FindSuffix(T)` is boundary set representable. The set of functions consistent with moving the cursor from position 1 to position 2 is concisely represented by the range `["s", "shall fight on the seas and oceans...in the air."]`.

Now, suppose we then observe the cursor moving from position 2 to position 3. This second movement is also consistent with a set of `FindSuffix` functions which can be represented with the range `["sh", "shall fight with growing confidence and growing strength in the air."]`. In this case, note that the set does not include "s", because the function `FindSuffix("s")` would have stopped right before "seas" instead of jumping all the way to position 3, so it is not consistent with the example.

Now we mentioned before that one important aspect of symbolic representations is the ability to manipulate them efficiently. In general, for boundary set representable version spaces, we can easily compute intersections as shown below:

$$[a_l, a_h] \cap [b_l, b_h] = [lub(a_l, b_l), glb(a_h, b_h)]$$

So in this case, if we want the set of programs that are consistent with both demonstrations, we can represent that set concisely as `["sh", "shall fight "]`, since "sh" is the shortest string that has both "s" and "sh" as a prefix, and "shall fight " is the longest common prefix of the two upper bounds.

6.3 Version Space Algebra

Lattice-based version spaces are great if you have them, but can also be very restrictive. The real power of the Version Space Algebra approach is the ability to symbolically represent complex compositions of simpler version spaces. For example, two powerful operations for combining version spaces are Union and Join explained below.

Union. The first form of composition is union $VS_{H_1,D} \cup VS_{H_2,D}$. As an example, consider the version space for `FindSuffix` presented earlier. There is an analogous `FindPrefix(T)` that finds the position in the text *after* the next occurrence of the string T. `FindPrefix` also has a lattice associated with it, except it is defined in terms of suffixes instead of prefixes. Now, we saw earlier that given two examples, one from the cursor from position 1 to 2, and one from 2 to 3, you could get the space `FindSuffix(["sh", "shall fight "])`. Well, by a similar process, we can see that the two examples are consistent with the space `FindPrefix(["we", ", we"])`. So the union of these two version spaces can be represented simply as

```
FindSuffix(["sh", "shall fight "]) U FindPrefix(["we", ", we"])
```

. If we added an additional example of jumping from the beginning of the text to position 1, the added example would have a version space of the form

```
FindSuffix(["shall f", "shall fight in France...air"]) U FindPrefix(["We",
     "We shall go on to the end. We"])
```

When we take the intersection of the two version spaces, we can take the intersection independently for the two kinds of version spaces and we get

```
FindSuffix(["shall f", "shall fight "]) ∪ ∅
```

We now have an empty set for FindPrefix, because there is no string that has both "we " and "We " as suffixes, so the glb of the two is $\top$.

Join. This is a form of relational join for version spaces defined as follows:

$$VS_{H_1,D_1} \bowtie VS_{H_2,D_2} = \{(h_1, h_2) | h_1 \in VS_{H_1,D_1}, h_2 \in VS_{H_2,D_2}, C((h_1, h_2), D)\}$$
$$\text{where}$$
$$D = \{(d_1^i, d_2^i)\} \text{ given } D_1 = \{d_1^i\} \text{ and } D_2 = \{d_2^i\}.$$

The function C in the definition is meant to stand for a consistency check that can be used to select which pairs in the cross product can actually be combined together.

In the case of PBD, Join can be used to describe sequences of operations together, especially when the demonstration allows us to separately provide evidence for the different actions in the sequence.

Continuing with our running example, suppose that each of the jumps of the cursor, from position 1 to position 2, to position 3 were treated as part of a sequence of actions, rather than independent examples corresponding to the same program. In that case, we could model this as a three way join of actions, and maintain a version space for each of the joins.

Overall, the key idea in the version space algebra framework is that we have symbolic representations of sets of progrmas. The representations may look like ASTs, but they are not ASTs representing programs. Instead, they are ASTs representing the algebraic operations over sets of programs that led to the current set.

6.4 Version Spaces meet E-Graphs in FlashFill

The general framework of version space algebras is ultimately only as powerful as the underlying representations used for the individual version spaces. The work of Lau and her collaborators built a complex space of text manipulation functions from lattice based representations and compositions thereof, but ultimately they were not very successful in providing sufficiently reliable automation. Around 2010, though, Sumit Gulwani and his collaborators revisited some of these ideas armed with more powerful program representations that enabled a careful language design aimed at a narrower class of text manipulations and developed the FlashFill system which was eventually incorporated into Excel Office [3] [4].

6.4.1 A language for text manipulation.

The first key ingredient in the flashfill system was the design of the language. The language is described in terms of two levels of abstraction. The first level consists of Trace Expressions defined according to the grammar below:

Trace expression	$e :=$	Concat($f_1, \ldots, f_n$) | f
Atomic expression	$f :=$	ConstStr(s) String constant s
		| SubStr(v_i, p_1, p_2) Produces a substring of v_i that lies between the positions indicated by p_1 and p_2
		| Loop($\lambda w.e$) Looping construct where e is the loop body and w the loop index.
Position	$p :=$	$CPos(k)$ Constant position k
		$Pos(r_1, r_2, c)$ The cth position that has r1 as prefix and r2 as suffix
Integer expr	$c :=$	$k \mid k_1 * w + k$ where w is a loop index
Regular expr	$r :=$	$T \mid TokenSeq(T_1, \ldots, T_k) \mid \epsilon$

The langauge above is very expressive. For example, in our running example, suppose I want to collect all phrases that appear between a "shall" and a punctuation mark, including the word shall. We could write such an expression as

$$\text{Loop}(\lambda w.\text{SubStr}(in, \text{Pos}(\text{“”}, \text{“}shall\text{”}, w), \text{Pos}(\text{“”}, PunctuationTok, w)))$$

The Loop construct iterates the expression until one of the terms becomes invalid. The expression itself finds the wth substring that starts with "shall" and ends right before a punctuation token. If we wanted to separate those with commas, we could use the program below instead:

$$\text{Loop}(\lambda w.\text{Concat}(\text{SubStr}(in, \text{Pos}(\text{“”}, \text{“}shall\text{”}, w), \text{Pos}(\text{“”}, PunctuationTok, w)), \text{“,”}))$$

On top of these trace expressions, FlashFill allows some control structure.

$$\begin{array}{rrl} \text{String program} & P := & \text{Switch}((b_1, e_1), \ldots, (b_n, e_n)) \mid f \\ \text{Boolean condition} & b := & d_1 \vee \ldots \vee d_n \\ \text{Conjunction} & d := & \pi_1 \wedge \ldots \wedge \pi_n \\ \text{Predicate} & \pi := & \text{Match}(v_i, r, k) \mid \neg\text{Match}(v_i, r, k) \end{array}$$

The Switch expression choses the first expression e_i whose corresponding condition b_i evaluates to true. The Match predicate will be true if there are k or more occurences of pattern r in string v_i.

Syntactic String Transformations: Example

Input v_1	Output
(425)-706-7709	425-706-7709
510.220.5586	510-220-5586
235 7654	425-235-7654
745-8139	425-745-8139

$Switch((b_1, e_1), (b_2, e_2))$, where

$$b_1 \equiv Match(v_1, NumTok, 3), \qquad b_2 \equiv \neg Match(v_1, NumTok, 3),$$

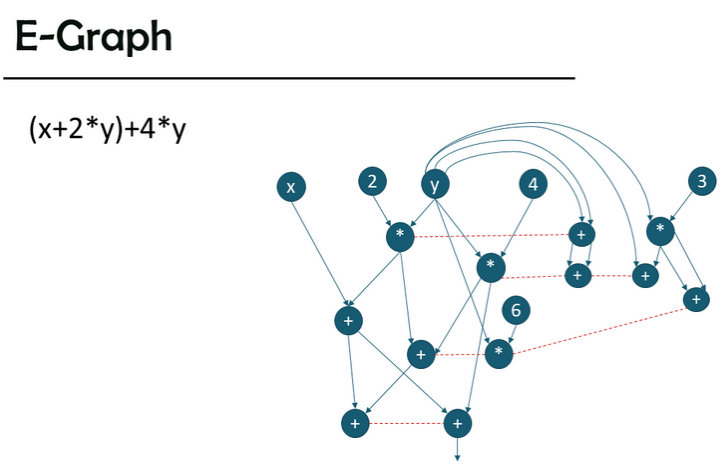
$$e_1 \equiv Concatenate\begin{pmatrix} SubStr2(v_1, NumTok, 1), \\ ConstStr(" - "), \\ SubStr2(v_1, NumTok, 2), \\ ConstStr(" - "), \\ SubStr2(v_1, NumTok, 3) \end{pmatrix} \quad e_2 \equiv Concatenate\begin{pmatrix} \text{ConstStr("425–")}, \\ ConstStr(" - "), \\ SubStr2(v_1, NumTok, 1), \\ ConstStr(" - "), \\ SubStr2(v_1, NumTok, 2) \end{pmatrix}$$

Slide by Sumit Gulwani

The key idea in the approach will be similar to the STUN approach we studied for bottom-up explicit search in that the goal is first to compute functions that work for some of the inputs, and then discover the conditional structure that combines programs that work for different subsets into a program that works for the whole input set. For the example in the figure, this involves independently discovering the programs that works for the cases with and without area code, and then discovering the condition that switches between the two. Note that the grammar is explicitly designed to support this approach and therefore only allows branches at the top level.

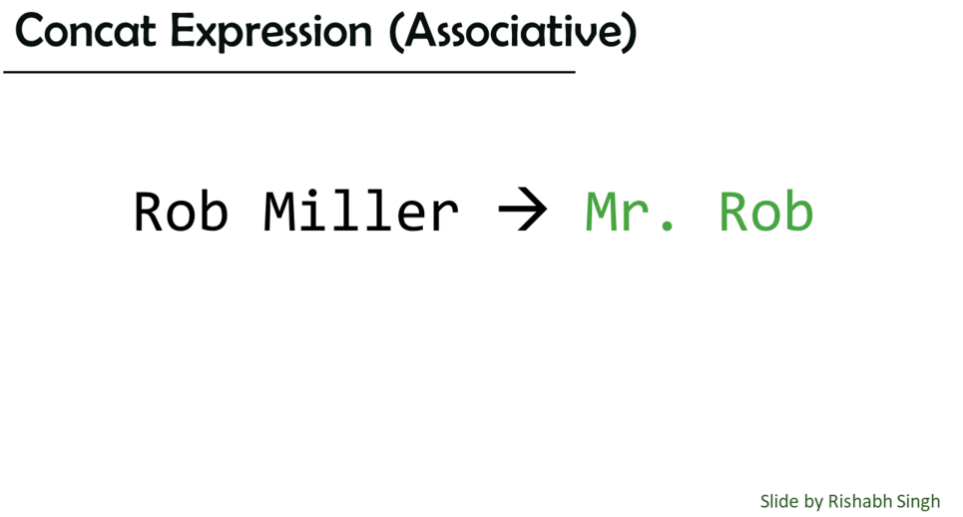
Beyond the choice of language, however, the major innovation in flashfill is in the representation of the version spaces for the trace expressions. By moving beyond lattice-based Boundary Set representations, flashfill was able to efficiently support the language described above. Before we dive into the details of the representation, though, we introduce an idea which is important for that representation, the idea of graph-based representations of sets of programs.

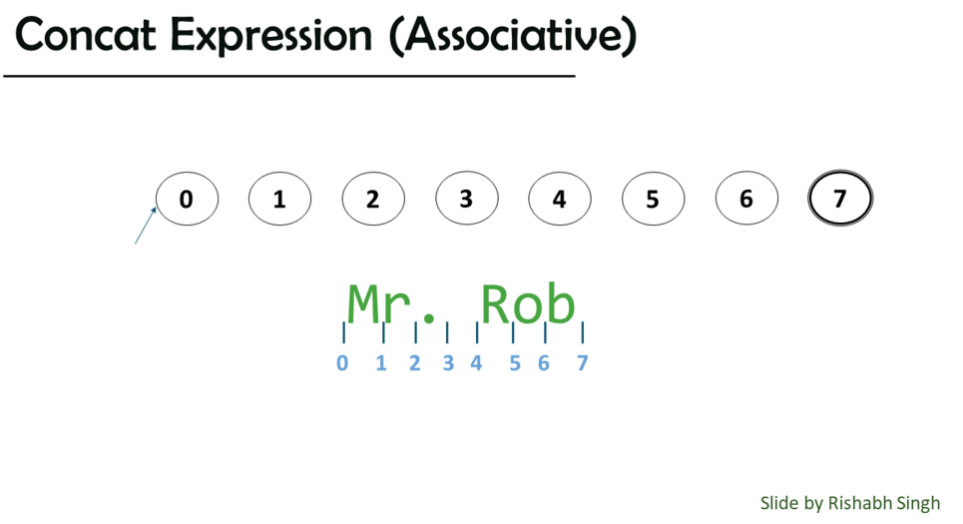
6.4.2 Graph representations for program spaces



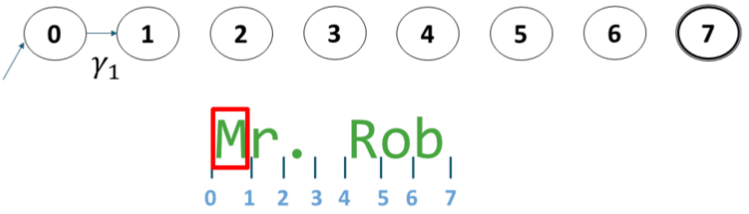
One of the key ingredients of FlashFill was the idea of using graphs to represent exponential spaces of programs. This idea traces its roots to a set of representations called E-graphs, first introduced by Downey, Sethi and Tarjan [5] and popularized by Nelson through applications ranging from theorem proving [6] to superoptimization [7]. The key idea in an e-graph is to represent an exponential set of possible programs with a compact graph, with different paths through the graph corresponding to different programs. For example, the animation on the right shows the construction of an E-graph that represents an exponential set of programs that can represent the expression $(x + 2 * y) + 4 * y$. The representation starts with an AST for the original expression, but is augmented by special edges that link nodes to equivalent nodes, for example, the node $2 * y$ is equivalent to the node $y + y$. Every combination of choices from the different equivalence classes corresponds to a distinct expression in the set. Just as with E-Graphs, the key data-structure in FlashFill is a graph that concisely represents an exponential set of possible expressions.

6.4.3 Representing Trace Expressions



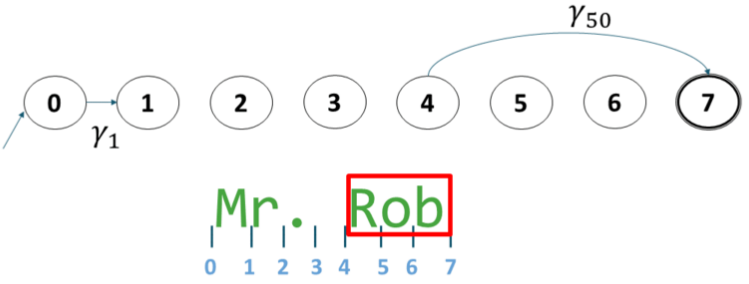


Concat Expression (Associative)

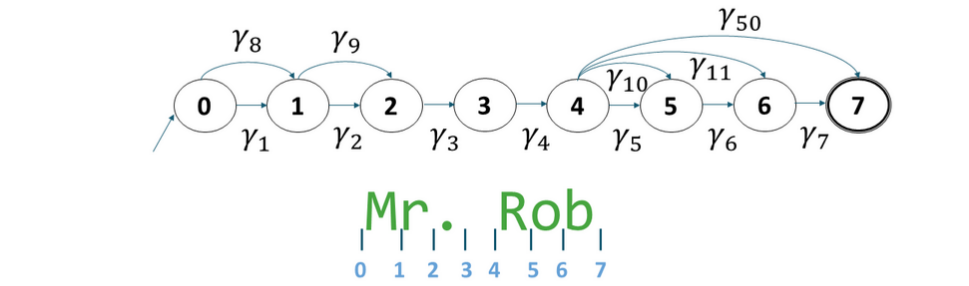


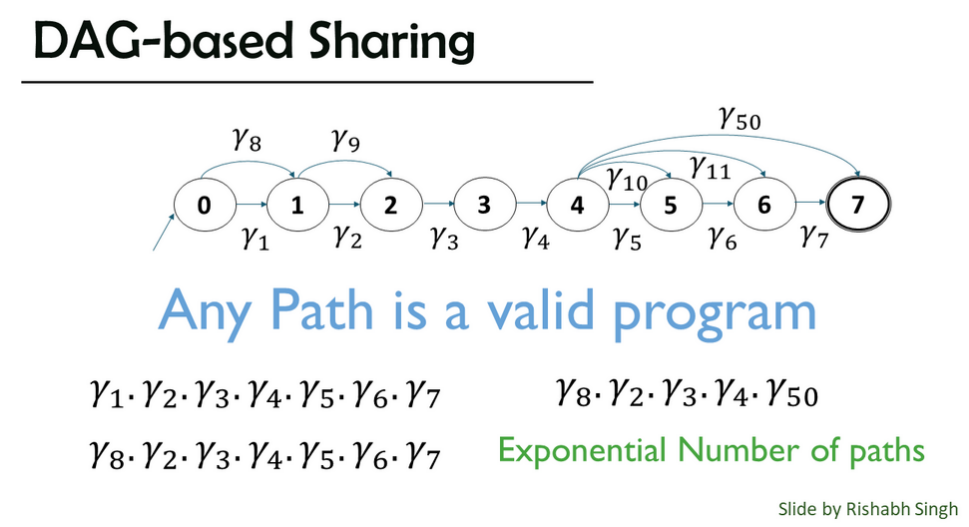
Slide by Rishabh Singh

Concat Expression (Associative)



Slide by Rishabh Singh

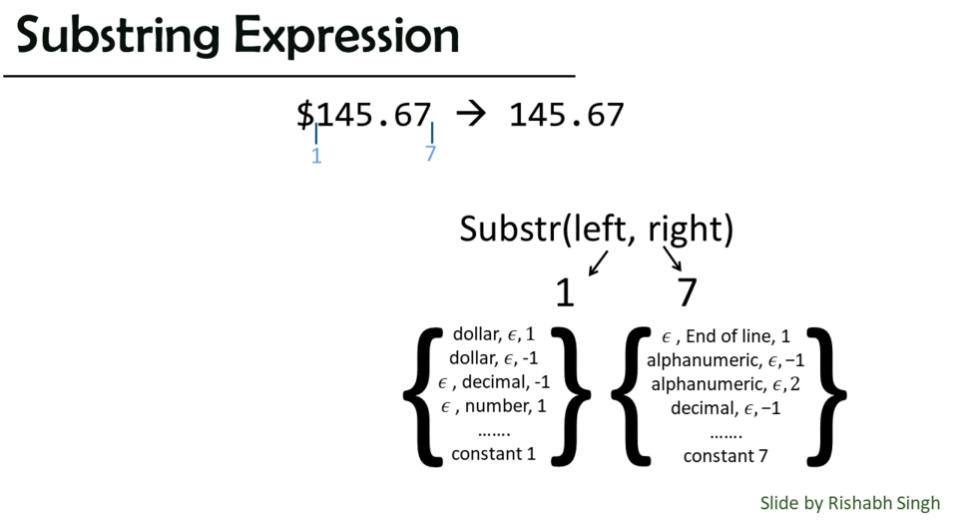




To understand the core idea in the representation, consider the example in the figure. The first key insight is that for every possible Trace Expression that could have generated that string, there is a partition of the string, where each sub-string in the partition was generated by its own atomic expression. For example, in one partition, each character was generated by an atomic expression γ_i. But in a different partition, maybe the entire string Rob was generated by an atomic expression γ_{50}. Just like in the e-graph example, we can add edges to the graph corresponding to possible atomic expressions that could have generated different sub-strings. And just like in the e-graph case, every edge we add leads to an exponential growth in the programs we are representing, because in this case, every path through this graph corresponds to a different program that could have represented the string.

This leaves us with two big challenges: how to efficiently represent the programs γ_i corresponding to each edge, and how to perform intersection between two such graphs efficiently.

6.4.4 Learning atomic expressions



The problem of learning atomic expressions is simpler than it looks thanks to the structure of the language. The first thing to note is that there are only three possibilities for an atomic expression: It may be a constant, it may be a substring, or it may be a loop expression. Constants are trivial to deal with, the more interesting ones are substring expression. The key observation is that the different patterns in the SubStr expression are independent of each other. For example, in the example from the figure, any pattern that matches at position 1 can be combined with any pattern that matches at position seven. The patterns themselves can actually be enumerated exhaustively given the limited scope of the pattern language. Identifying loops simply boils down to identifying repeating patterns in the γ_i that can be merged into a common expression using antiunification.

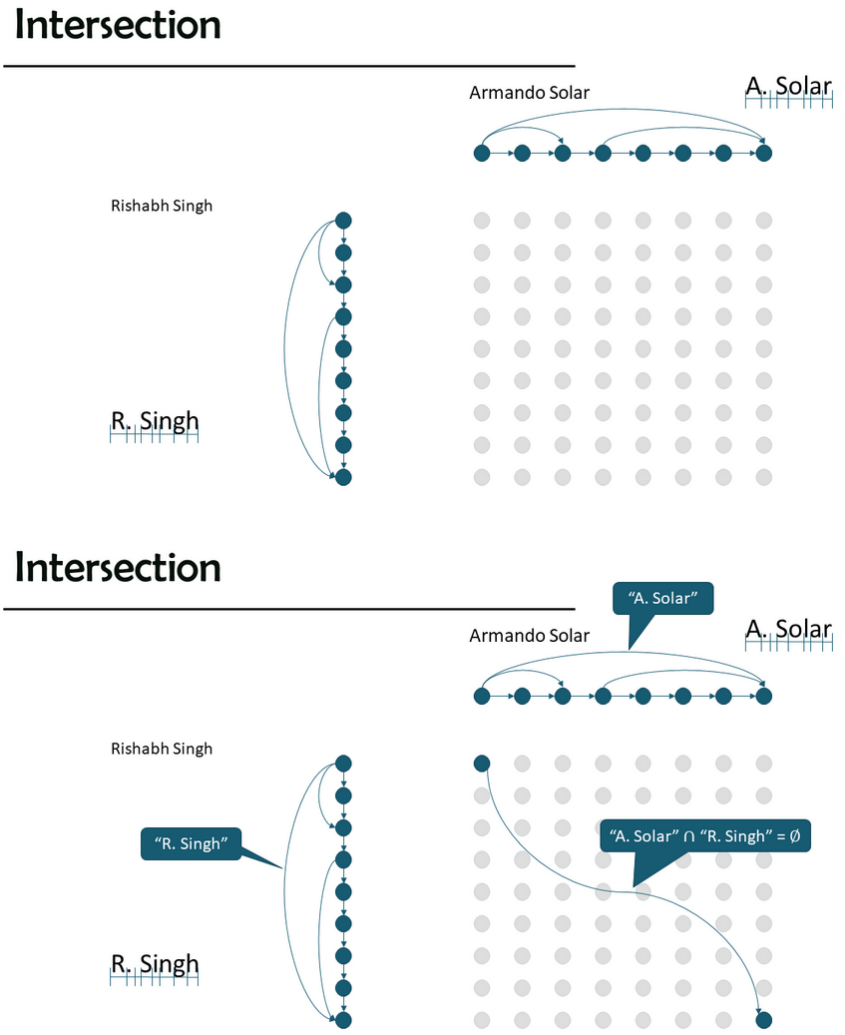
6.4.5 Intersection

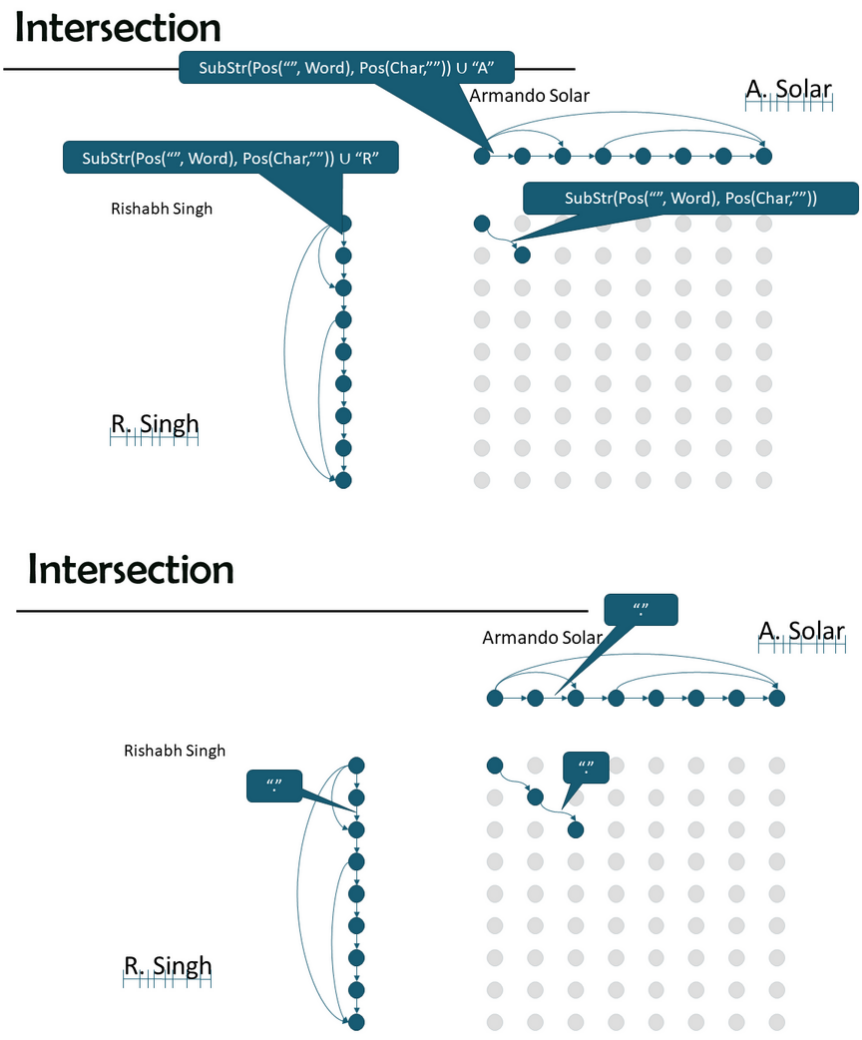
Given two graphs representing a set of trace expressions, it is relatively easy to compute a new graph that represents its intersection.

At a high-level, given two graphs $G_1 = (N_1, E_1)$ $G_2 = (N_2, E_2)$ where N_i are the sets of nodes and $E_i = \{(s_i^t, d_i^t, \gamma_i^t)\}$ are sets of edges from a source node s_i^t to a destination node d_i^t labeled with a set of atomic expressions γ_i^t, we can compute a graph for the intersection as follows:

$$
\begin{aligned}
G_1 \cap G_2 &= (N, G) \text{ where} \\
N &= N_1 \times N_2 \\
G &= \{((s_1^t, s_2^v), (d_1^t, d_2^v), \gamma_1^t \cup \gamma_2^v) | (s_1^t, d_1^t, \gamma_1^t) \in E_1 \wedge (s_2^v, d_2^v, \gamma_2^v) \in E_2\}
\end{aligned}
$$

In other words, the graph representing the intersection will have a node for each pair of nodes from the two graphs, and for every pair of edges, there will be an edge representing their intersection.





The figure illustrates this intersection process. In the figure we can see how select pairs of edges from the two original graphs get intersected. For example, there is an edge in one graph corresponding to the whole thing being a single string, but it's intersection with another edge that also involves a constant string leads to an empty set.

On the other hand, the intersection between an edge that includes a substring from before the first word to after the first character intersects with a similar edge from another example and leads to an edge with the expression `SubStr(in, Pos("", Word, 1), Pos(Char,"", 1))` . The indices in the Pos expression are omitted in the figure for clarity.

This basic approach has been extended to a number of different domains. More recently, Alex Polozov and Sumit Gulwani, have developed a framework

called Prose (also called FlashMeta in the literature) that makes it possible to easily build these kind of representations for other domains [8].

References

[1] Tessa Lau, Steven A. Wolfman, Pedro Domingos, and Daniel S. Weld. Programming by demonstration using version space algebra. *Mach. Learn.*, 53 (1-2):111–156, oct 2003. ISSN 0885-6125. doi: 10.1023/A:1025671410623. URL https://doi.org/10.1023/A:1025671410623.

[2] Tom M. Mitchell. Generalization as search. *Artif. Intell.*, 18(2):203–226, 1982. doi: 10.1016/0004-3702(82)90040-6. URL https://doi.org/10.1016/0004-3702(82)90040-6.

[3] Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '11, pages 317–330, New York, NY, USA, 2011. ACM. ISBN 978-1-4503-0490-0. doi: 10.1145/1926385.1926423. URL http://doi.acm.org/10.1145/1926385.1926423.

[4] Rishabh Singh and Sumit Gulwani. Learning semantic string transformations from examples. *PVLDB*, 5(8):740–751, 2012. doi: 10.14778/2212351.2212356. URL http://vldb.org/pvldb/vol5/p740_rishabhsingh_vldb2012.pdf.

[5] Peter J. Downey, Ravi Sethi, and Robert Endre Tarjan. Variations on the common subexpression problem. *J. ACM*, 27(4):758–771, October 1980. ISSN 0004-5411. doi: 10.1145/322217.322228. URL http://doi.acm.org/10.1145/322217.322228.

[6] David Detlefs, Greg Nelson, and James B. Saxe. Simplify: a theorem prover for program checking. *J. ACM*, 52(3):365–473, 2005. doi: 10.1145/1066100.1066102. URL http://doi.acm.org/10.1145/1066100.1066102.

[7] Rajeev Joshi, Greg Nelson, and Keith Randall. Denali: A goal-directed superoptimizer. In *Proceedings of the ACM SIGPLAN 2002 Conference on Programming Language Design and Implementation*, PLDI '02, pages 304–314, New York, NY, USA, 2002. ACM. ISBN 1-58113-463-0. doi: 10.1145/512529.512566. URL http://doi.acm.org/10.1145/512529.512566.

[8] Oleksandr Polozov and Sumit Gulwani. Flashmeta: a framework for inductive program synthesis. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2015, part of SPLASH 2015, Pittsburgh, PA, USA, October 25-30, 2015*, pages 107–126, 2015. doi: 10.1145/2814270.2814310. URL http://doi.acm.org/10.1145/2814270.2814310.